UNIVERSIDAD PRIVADA DEL NORTE

INGENIERÍA DE SOFTWARE

eXpande - Tema 5

Retos de análisis: Modelado Conceptual y Diseño con Diagramas de Clases

Semanas 11 y 12

Propósito: analizar, cuestionar y mejorar decisiones de modelado UML mediante casos breves y situaciones problemáticas.

Curso: Ingeniería de Software - CIIN1252P

Periodo académico: 2026-2

1. Indicaciones

Los cinco retos están diseñados para complementar las Semanas 11 y 12. No se busca memorizar símbolos UML, sino explicar por qué una decisión de modelado es adecuada o necesita ser corregida.

- Lea cada caso antes de responder.
- Justifique siempre sus decisiones con conceptos trabajados en clase: dominio, responsabilidades, trazabilidad, multiplicidades, capas, secuencia, cohesión y acoplamiento.
- Cuando detecte un error, no basta con señalarlo: explique por qué es un problema y proponga una corrección.
- Use un lenguaje técnico sencillo y claro.
- En los retos de diseño, considere también excepciones y necesidades de usuarios que podrían quedar fuera del flujo ideal.

Criterio de trabajo:

Una respuesta es sólida cuando puede relacionarse con requisitos, casos de uso y responsabilidades verificables, y cuando mantiene coherencia entre los diferentes artefactos.

2. Recordatorio breve

Concepto	Idea clave
Modelo conceptual	Representa conceptos relevantes del dominio: qué existe y cómo se relaciona.
Modelo de diseño	Organiza responsabilidades de la solución según una arquitectura.
View / Interfaz	Recoge datos y muestra resultados al usuario.
Controller	Recibe la solicitud y deriva el proceso.
Service	Coordina reglas y responsabilidades del caso de uso.
Repository	Consulta o persiste información.
Diagrama de secuencia	Muestra cómo colaboran los participantes para realizar un caso de uso.
Trazabilidad	Permite demostrar qué requisito o caso de uso justifica una clase, relación u operación.

Reto 1. Detectar y corregir decisiones de modelado

Caso: Registro de atención en un centro de salud

Un equipo debe construir el primer modelo conceptual para el caso de uso Registrar atención. El sistema registra pacientes, profesionales de salud y atenciones. Una atención pertenece a un paciente y es realizada por un profesional de salud.

El equipo propone las siguientes afirmaciones:

Código	Afirmación
A	Paciente debe ser una clase del dominio porque tiene identidad e información propia.
B	RegistrarAtencionView debe incluirse como clase del dominio porque el usuario la utiliza para registrar datos.
C	La relación Paciente-Atencion puede expresarse como: un Paciente tiene 0..* Atenciones y cada Atencion corresponde a 1 Paciente.
D	Atencion debe estar respaldada por un requisito o caso de uso; de lo contrario, debe revisarse si realmente pertenece al modelo.
E	BotonGuardar debe modelarse como clase del dominio porque ejecuta la operación de registro.
F	La matriz Requisito-Clase ayuda a detectar clases sin justificación y requisitos sin representación.

Tarea:

- Identifique las afirmaciones que considere incorrectas.
- Explique por qué son incorrectas.
- Proponga una versión corregida para cada una.
- Indique qué principio de modelado está aplicando en su corrección.

Reto 2. Completar un modelo conceptual

Caso: Asociación de productores de cacao

Una asociación registra a sus productores, los lotes de cacao disponibles y los pedidos realizados por compradores. Un pedido puede incluir cacao proveniente de varios lotes. Para cada lote comprometido se debe registrar la cantidad asignada al pedido.

Información disponible:

Requisito	Descripción
RF-01	Registrar productores asociados.
RF-02	Registrar lotes de cacao por productor.
RF-03	Registrar pedidos de compradores.
RF-04	Asignar uno o más lotes a un pedido indicando la cantidad comprometida.
RF-05	Actualizar la cantidad disponible de cada lote después de confirmar el pedido.

Modelo incompleto propuesto por el equipo:

Clase candidata	Observación inicial
Productor	Tiene identidad propia y registra lotes.
Lote	Tiene cantidad disponible y pertenece a un productor.
Pedido	Representa la solicitud del comprador.
¿Clase faltante?	Debe permitir registrar cuánto de cada lote se asigna a un pedido.
PantallaPedido	El equipo duda si debe formar parte del modelo conceptual.

Tarea:

- Identifique la clase faltante y justifique por qué es necesaria.
- Indique qué relaciones y multiplicidades propondría entre Productor, Lote, Pedido y la clase faltante.
- Decida si PantallaPedido debe formar parte del modelo conceptual y explique su respuesta.
- Construya una pequeña matriz Requisito-Clase para RF-02, RF-04 y RF-05.

Reto 3. Comparar dos diseños por capas

Caso: Reserva de menú en un comedor universitario

Un estudiante selecciona un menú, solicita una reserva y recibe una confirmación. El sistema debe verificar cupos antes de guardar la reserva.

Dos equipos presentan estas alternativas:

Aspecto	Diseño A	Diseño B
Flujo principal	View -> Repository	View -> Controller -> Service -> Repository
Validación de cupo	La realiza la View	La coordina ReservaService
Persistencia	La View guarda directamente la reserva	ReservaRepository guarda la reserva
Controller	No se utiliza	Recibe la solicitud y delega
Entidades de dominio	Menu y Reserva	Menu y Reserva

Tarea:

- Seleccione el diseño que considere más coherente para el caso.
- Justifique su elección usando separación de responsabilidades, cohesión y acoplamiento.
- Identifique una debilidad que todavía podría existir incluso en el diseño seleccionado.
- Proponga una mejora concreta.

Reto 4. Detectar inconsistencias entre artefactos

Caso: Registrar atención - revisión del diagrama de secuencia

El equipo ya tiene las clases Atencion, Paciente, AtencionController, AtencionService, PacienteRepository y AtencionRepository. Luego construye la siguiente secuencia simplificada:

Orden	Origen	Destino	Mensaje
1	ProfesionalSalud	AtencionController	registrarAtencion(datos)
2	AtencionController	AtencionService	registrar(datos)
3	AtencionService	PacienteRepository	buscarPaciente(id)
4	AtencionService	AtencionRepository	guardar(atencion)
5	AtencionRepository	RegistrarAtencionView	mostrarConfirmacion()

Información adicional:

- El diseño de la clase AtencionRepository solo declara la operación guardar(atencion).
- RegistrarAtencionView existe en el diseño, pero no aparece al inicio de la secuencia.
- El actor ProfesionalSalud interactúa con una interfaz en el sistema real.

Tarea:

- Identifique las inconsistencias entre la secuencia y el diseño.
- Explique por qué cada inconsistencia puede generar confusión o pérdida de trazabilidad.
- Proponga una secuencia corregida, indicando al menos los participantes y mensajes principales.
- Explique qué operación o responsabilidad debe existir en cada receptor del mensaje.

Reto 5. Refactorizar y considerar excepciones reales

Caso: Atención de emergencia con conectividad limitada

En un centro de salud rural, el equipo implementó un AtencionService que realiza todas estas tareas: valida identidad, consulta pacientes, registra atenciones, controla medicamentos, genera reportes, envía notificaciones y decide qué hacer cuando no existe conexión a Internet.

Durante las pruebas se detectan dos situaciones:

- Algunos pacientes llegan sin DNI o sin documento disponible.
- La conexión a Internet puede interrumpirse durante una atención de emergencia.

Tarea:

- Analice si AtencionService mantiene una responsabilidad clara o si presenta baja cohesión.
- Identifique al menos dos responsabilidades que podrían separarse o reorganizarse.
- Proponga una refactorización sencilla indicando qué componentes o servicios utilizaría.
- Explique cómo cambiaría el diagrama de secuencia para manejar la falta de conectividad.
- Explique qué decisión evitaría excluir a un paciente que no presenta DNI.
- Registre una decisión de diseño con el formato: problema -> alternativas -> criterio -> decisión -> impacto.

3. Cierre reflexivo

Después de resolver los cinco retos, responda brevemente:

- ¿Qué diferencia considera más importante entre modelo conceptual y modelo de diseño?
- ¿Qué error de modelado cree que puede afectar más la mantenibilidad del sistema?
- ¿Por qué un diagrama de secuencia puede revelar problemas que no eran visibles en el diagrama de clases?
- ¿Cómo puede una decisión de diseño excluir a ciertos usuarios aunque el software sea técnicamente correcto?

_____

_____

_____

_____

_____

Idea clave

Modelar no significa dibujar más diagramas. Significa tomar decisiones justificadas y mantener coherencia entre necesidades, requisitos, clases, responsabilidades e interacciones.

4. Referencias

- Object Management Group (OMG). (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1
- ISO/IEC/IEEE. (2018). ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering. IEEE Standards Association. https://standards.ieee.org/ieee/29148/6937/
- Pressman, R. S., & Maxim, B. R. (2021). Ingeniería de software: un enfoque práctico. McGraw-Hill.
- Jiménez de Parga, C. (2021). UML: arquitectura de aplicaciones en Java, C++ y Python. eLibro.
- Universidad Privada del Norte. (2026). Sílabo del curso Ingeniería de Software CIIN1252P, periodo 2026-2.
- Universidad Privada del Norte. (2026). Diseño de sesión de clase - Tema 5: Modelado Conceptual con Diagramas de Clases.